

从后续纠正学会第一次回答

面向单个用户的离线到在线提示蒸馏与 LoRA 更新

中文算法设计笔记 2026 年 8 月 29 日

1 先说结论

单纯的“提示词版本”已经有人做过。DSO 从日志反馈学习提示词策略，AGP 优化个性化推荐提示词，GEPA 根据执行轨迹改写提示词 [2, 3, 4]。因此，不能把“冻结模型，只优化提示词”写成新的研究点。

现有的 18 动作方案也不适合当前数据。它先把用户偏好压成

$$a = (m, \tau, s), \quad |\mathcal{A}| = 18,$$

再由手写的 $\text{Compile}(a, x)$ 生成指令。真实数据却包含自由文本回答、JSON 工具调用、工具返回和多步状态。18 个标签既不能表示这些动作，也不能从数据中学出新的行为。

更直接的做法是：

用户在后续一轮指出问题后，把这条纠正配回产生错误之前的状态。教师模型在“原状态 + 纠正”下给出方向；用户 LoRA 在不看纠正的原状态上学习这个方向。部署后，模型在第一次回答时就应使用该偏好。

本文把这一步简称为**纠正前移**。提示词仍然有用，但只在训练时给教师提供纠正。部署对象是每位用户自己的 LoRA，不是一份不断变长的提示词。

2 真实数据告诉了我们什么

2.1 当前可用数据

表 1: /fsx/xzhichao/workspace/oocso/data 中与本设计直接相关的文件。

文件	规模	用途
tasks_train.jsonl	120 个任务	工具调用训练任务；当前 run.py 取编号 0-79 做离线训练
tasks_dev.jsonl	30 个任务	开发阶段检查任务成功率
tasks_eval.jsonl	30 个任务	最终保留任务，即 X_{holdout}
dump_samples.json	3 个离线会话、2 个在线会话	展示真实字段和当前失败模式，不足以支持最终结论
preview_pipeline.json	4 个任务、2 条离线线程、3 个在线记录	小规模流程预览
preview_pi0_student.pt	288 个 LoRA 张量	已训练的用户适配器快照

工具任务分为七类：

$$C = \{\text{single_tool}, \text{two_step_chain}, \text{chain3}, \text{cross_rate}, \text{cross_rate3}, \text{sum_two}, \text{error_correction}\}.$$

每条任务给出允许使用的工具、答案、误差范围和最大步数。它们可以检查任务是否真的完成，但本身不包含用户偏好。偏好信号来自同一用户的对话和纠正，不能从工具任务中凭空补写。

2.2 已经观察到的结构性问题

dump_samples.json 中的三条离线会话具有相同结构：

1. 第一次回答没有看到偏好提示，回答带有加粗、编号或小标题，奖励为 -1 ；
2. 用户要求去掉这些格式；
3. 第二次回答看到明确纠正后改成纯文本，奖励为 $+1$ 。

因此，所有正样本都带有明确纠正。普通 GRPO 只能学到“用户提醒后再改”。已有 π_0 在新任务的第一次回答上仍然违规，这与数据结构一致。问题不在于训练轮数太少，而在于正样本处于错误的状态。

数据还暴露了两个实现问题：有的用户反应没有结束思考块，因而可见文本为空；有的 hint 只剩一个字“和”。这两种记录不能进入纠正前移。

3 模型、输入和输出

3.1 训练什么

冻结主干为

$$\text{FA}_{\phi} = \text{Qwen3-4B-Thinking-2507}.$$

每位用户 u 有一个 LoRA 适配器 $\Delta\theta_u$ 。现有检查点对应

$$r = 16, \quad \alpha = 32,$$

并插入每层注意力的 `q_proj, k_proj, v_proj, o_proj`。检查点包含 288 个矩阵和 11,796,480 个可训练数。主干参数、工具和环境都不更新。

这里不再额外训练一个 18 类分类器。用户身份由加载哪个 LoRA 决定，所以单用户实验也不需要用户 ID embedding。

3.2 输入是什么

一次决策前的模型输入记为 x 。它是代码中已经存在的 `state_messages/state`:

$$x = (\text{system}, \text{用户任务}, \text{此前回答}, \text{此前工具返回}).$$

输入只包含本次决策前已经发生的内容。评测第一次回答时， x 不含当前任务的用户纠正，也不含人工写入的偏好提示。

3.3 输出是什么

输出 $a = (a_1, \dots, a_L)$ 是模型实际生成的 token 序列:

- 对普通对话， a 是用户看到的回答;
- 对工具任务， a 是一个 `<tool_call>{...}</tool_call>` JSON 调用;
- 多步任务每一步重新形成状态 x_t ，再生成下一条 a_t 。

策略为

$$\pi_{\theta}(a \mid x) = \prod_{j=1}^L \pi_{\phi, \Delta\theta_u}(a_j \mid x, a_{<j}). \quad (1)$$

动作空间 $\mathcal{A}(x)$ 是当前协议允许的 token 序列，不是固定的 18 个标签。

4 与原 PDF 的符号对应

表 2: 保留原符号，但改变错误的动作定义。

符号	本文含义	与原方案的关系
D_{hist}	同一用户按时间排列的完整会话	保留
$D_{\text{train}}, D_{\text{holdout}}$	过去会话与未来保留会话	保留；必须按完整会话和时间切分
X_{holdout}	最终工具任务和未来用户任务	保留；不参与训练、hint 提取或选参
$\tilde{X}$	可选模拟任务	首个实验设为 $\emptyset$ ，先使用真实任务
C, c	七类工具任务及之后明确加入的用户任务类	保留；类别来自数据，不由 Judge 临时发明
P_u	可选的用户偏好摘要	只给教师、Judge 和提示词基线；适配器评测不输入它
$a, \mathcal{A}(x)$	实际回答或工具调用 token 序列	替换 (m, τ, s) 和 18 动作
y_a	执行 a 后的工具返回、最终答案或用户反应	保留为可观察结果
r_a	由任务结果和真实偏好证据得到的分数	保留，但不由手写动作表产生
$A_a = r_a - \bar{r}$	同一状态候选动作的相对优势	保留
$\pi_\theta, \pi_{\text{base}}$	带用户 LoRA 的策略与不带 LoRA 的主干策略	保留
FA_ϕ	冻结主干、工具协议和执行环境	保留

$\text{Compile}(a, x)$ 被删除。模型输出本身就是可执行动作，不再把标签翻译成手写提示词。

5 从历史中构造纠正记录

对一段历史，保留五项：

$$e_i = (x_i, a_i^-, f_i, a_i^+, o_i). \quad (2)$$

其中 x_i 是第一次错误回答前的状态， a_i^- 是该回答， f_i 是用户随后给出的纠正， a_i^+ 是纠正后的回答， o_i 是任务结果或后续用户反应。

从 f_i 提取一句具体纠正：

$$h_i = \text{ExtractCorrection}(a_i^-, f_i, a_i^+). \quad (3)$$

提取器只能整理用户已经说出的内容，不能补写新的偏好。以下记录丢弃：

- f_i 的可见文本为空；

- h_i 过短、无法执行或只包含泛泛评价；
- a_i^+ 仍未被接受；
- 在有确定性检查器的任务上，纠正后的结果仍然错误。

若 a_i^+ 脱离上一轮后仍能独立回答原任务，就直接把它作为参考序列 $\hat{a}_i$ 。否则，让冻结教师在 $x_i \oplus h_i$ 下重新生成 $\hat{a}_i$ ，并再次检查结果。这样不会把“我已经改好了”一类依赖上文的句子当成第一次回答。

6 离线算法：把纠正前移

6.1 构造只差一条纠正的两个输入

对每条有效记录，构造

$$x_i^0 = x_i, \quad x_i^h = x_i \oplus h_i. \quad (4)$$

两个输入的任务、历史和工具状态完全相同，唯一差别是教师输入多了用户纠正。

在参考序列 $\hat{a}_i$ 的第 j 个位置，记录旧策略在两个输入上的分布：

$$p_{ij}^0(v) = \pi_{\theta_{\text{old}}}(v \mid x_i^0, \hat{a}_{i,<j}), \quad (5)$$

$$q_{ij}(v) = \text{stopgrad} \left[\pi_{\theta_{\text{old}}}(v \mid x_i^h, \hat{a}_{i,<j}) \right]. \quad (6)$$

教师和学生使用同一个当前检查点；教师分支停止梯度。纠正带来的 token 方向为

$$d_{ij}(v) = \text{clip}(\log q_{ij}(v) - \log p_{ij}^0(v), -C, C). \quad (7)$$

共同的任务知识在相减时大体抵消，留下的是纠正改变了哪些 token 的概率。

6.2 在没有纠正的输入上学习这个方向

只在教师和学生 top- k token 的并集 S_{ij} 上计算，避免遍历整个词表。定义

$$w_{ij}(v) = \text{softmax}_{v \in S_{ij}} \log p_{ij}^0(v), \quad (8)$$

$$A_{ij}^{\text{corr}}(v) = w_{ij}(v) d_{ij}(v), \quad (9)$$

$$\rho_{ij}(v) = \frac{\pi_{\theta}(v \mid x_i^0, \hat{a}_{i,<j})}{p_{ij}^0(v)}. \quad (10)$$

纠正前移损失为

$$\mathcal{L}_{\text{corr}} = \sum_{i,j} \sum_{v \in S_{ij}} \max[-A_{ij}^{\text{corr}}(v) \rho_{ij}(v), -A_{ij}^{\text{corr}}(v) \text{clip}(\rho_{ij}(v), 1 - \epsilon, 1 + \epsilon)]. \quad (11)$$

这一步与 OpenClaw-RL 的 hint 蒸馏有直接关系 [1]。本文增加的是离线历史重建：从多轮日志找回 x_i^0 ，再把后续纠正配回这个状态。OPD 本身不是新贡献。

6.3 同时学习真实任务结果

纠正前移只告诉模型如何使用偏好，不能替代任务训练。对同一状态 x ，保留或采样一组真实动作：

$$\mathcal{A}_x = \{a^-, \hat{a}, a^{(1)}, \dots, a^{(K)}\}.$$

执行每个动作得到

$$y_a = \text{FA}_\phi.\text{Execute}(x, a).$$

任务分数 $r_a^{\text{task}} \in \{-1, +1\}$ 优先使用环境检查器。偏好分数 $r_a^{\text{pref}} \in [-1, 1]$ 来自真实用户反应、可复现规则或 Judge。合成分数为

$$r_a = r_a^{\text{task}} + \lambda_{\text{pref}} r_a^{\text{pref}}, \quad 0 \leq \lambda_{\text{pref}} < 1. \quad (12)$$

因此，任务失败的动作不会靠写作风格超过任务成功的动作。报告时仍分别列出两项分数。

在同一状态的候选组内计算

$$\bar{r}_x = \frac{1}{|\mathcal{A}_x|} \sum_{a \in \mathcal{A}_x} r_a, \quad A_a = r_a - \bar{r}_x, \quad (13)$$

再用现有的 clipped GRPO 得到 $\mathcal{L}_{\text{outcome}}$ 。

6.4 离线总目标

$$\mathcal{L}_{\text{offline}} = \lambda_{\text{corr}} \mathcal{L}_{\text{corr}} + \lambda_{\text{out}} \mathcal{L}_{\text{outcome}}. \quad (14)$$

第一项把后续纠正学到早先状态；第二项保住答案和工具调用能力。两项都只更新 $\Delta\theta_u$ 。

7 在线算法：继续用同一条更新

离线训练得到 π_0 。部署后，对每次交互保存

$$(x_t, a_t, y_t, f_t).$$

若用户满意或任务成功，记录结果供 outcome 更新使用。若用户纠正：

1. 冻结纠正发生前的 x_t ；
2. 从 f_t 提取 h_t ；
3. 构造 $x_t^0 = x_t$ 和 $x_t^h = x_t \oplus h_t$ ；
4. 生成并检查参考动作 $\hat{a}_t$ ；
5. 用式 (11) 在 x_t^0 上更新 LoRA；
6. 同一批新轨迹用式 (12) 做 outcome 更新。

在线阶段没有新的动作空间，也没有第二套优化器逻辑：

$$\theta_{t+1} = \theta_t - \eta_t \nabla_{\theta} \left(\lambda_{\text{corr}} \mathcal{L}_{\text{corr}}^{(t)} + \lambda_{\text{out}} \mathcal{L}_{\text{outcome}}^{(t)} \right). \quad (15)$$

区别只在数据：离线读取过去日志，在线读取刚发生的交互。

8 完整工作流

8.1 离线

1. 按完整会话和时间把 D_{hist} 切为 D_{train} 与 D_{holdout} 。先切分，再提取 P_u 或 hint。
2. 从 D_{train} 提取式 (2) 的纠正记录。
3. 过滤空反馈、损坏 hint 和未通过任务检查的记录。
4. 对每条记录构造 $x^0, x^h, \hat{a}$ 。
5. 用式 (11) 训练用户 LoRA。
6. 在真实工具任务上采样动作并执行，用 GRPO 保住任务能力。
7. 以固定训练预算得到 π_0 。训练过程中不读 D_{holdout} 或 X_{holdout} 。

8.2 在线

1. 加载同一用户的 π_0 。
2. 先回答或执行任务，再读取真实 next state。
3. 有纠正时，把纠正配回原状态；没有纠正时只保留结果信号。
4. 累积到一个小批量后继续更新同一个 LoRA。
5. 按预定间隔在开发任务上检查第一次回答和任务成功率。
6. 最后只在一次冻结的评测中读取 $D_{\text{holdout}}, X_{\text{holdout}}$ 。

9 如何使用现有 OOCSSO 数据

表 3: 第一轮可执行实验，不额外编造数据。

数据	进入算法的位置	说明
完整偏好会话	$\mathcal{L}_{\text{corr}}$	现有 dump 只用于单元测试；正式训练需要保存完整、可见的用户纠正
80 个离线工具任务	$\mathcal{L}_{\text{outcome}}$	沿用当前 <code>run.py</code> 的编号切分
40 个训练集中后段任务	在线训练或开发实验	在运行前固定用途，不能一边看结果一边改切分
30 个 dev 任务	开发检查	检查任务能力是否因个性化更新下降
30 个 eval 任务	X_{holdout}	最终只运行一次
$\tilde{X}$	暂不使用	当前已有真实任务，先避免模拟问题引入新的偏好假设

偏好实验和工具实验承担不同职责。偏好会话提供“用户希望怎样做”，工具任务提供“任务有没有完成”。两者通过式 (14) 联合训练，但不能把工具任务自动标成某种用户偏好。

10 必须比较的版本

表 4: 最小对照。

版本	回答的问题
π_{base}	不做个性化时表现如何？
$\pi_{\text{base}} + P_u$	每次把偏好放进提示词是否已经足够？
普通离线 GRPO	只用历史奖励、不给纠正前移时能学到什么？
普通离线 SFT/DPO	把纠正后回答当训练目标是否足够？
Online-only	没有离线起点时需要多少次实时纠正？
OpenClaw-RL/OPD	
本文：离线纠正前移 → 在线继续更新	后续纠正是否能提前作用于第一次回答？

至少分开报告：

- 第一次回答符合偏好的比例，以及连续三次符合所需的会话数；
- 工具任务成功率、平均步数和工具调用错误率；

- 各任务类别结果；
- 每位用户 LoRA 的大小、训练时间和在线更新次数。

不要把个性化和任务成功压成一个总分。

11 创新边界

以下部分已有工作已经覆盖：

- 提示词优化：DSO、AGP 和 GEPA [2, 3, 4]；
- 每位用户一个离线 LoRA：OPPU [5]；
- 持续更新用户 LoRA：SPRInG [6]；
- 从 next state 提取 reward 和 hint，并做在线 OPD：OpenClaw-RL [1]；
- 离线到在线训练的一般过渡：已有半在线 DPO/RL 工作 [7]。

因此，本文不能声称首次做提示词个性化、首次训练用户 LoRA，或首次使用 hint 蒸馏。当前可检验的差异是：

从同一用户的历史中重建纠正前状态，把后来出现的纠正蒸馏回该状态，并在离线和部署后使用同一条更新，同时用可执行任务结果约束动作。

这仍是待验证的方法设计。实验成功和更完整的文献核对之前，不应写成已经成立的新算法。

12 对现有代码的最小改动

表 5: 实现位置。本文只给设计，不在本次改动这些代码。

文件	需要的后续改动
<code>pref/session.py</code>	保存纠正前状态、可见用户反馈、纠正后回答和是否被接受
<code>pref/pref_prm.py</code>	只从可见反馈提取 hint；继续过滤过短或损坏候选
<code>algo.py</code>	让 OPD 沿检查通过的参考序列计算，并使用 teacher/student top- k 并集
<code>pref/run_pref.py</code>	离线阶段加入纠正前移，不再只对带纠正的后续状态做 GRPO
<code>agent.py</code> 与 <code>env/</code>	保持现有工具执行和确定性任务检查，不改工具协议

第一项单元测试应直接使用现有苹果题记录：训练样本的 student 输入只能是第一次回答前的 `state_messages`，不能包含“别用格式”；teacher 输入才可以多出该纠正。若这一点没有通过，算法仍会退回当前的“提醒后才改”。

参考文献

- [1] Yinjie Wang, Xuyang Chen, Xiaolong Jin, Mengdi Wang, and Ling Yang. OpenClaw-RL: Train Any Agent Simply by Talking. [arXiv:2603.10165](#), 2026.
- [2] Haruka Kiyohara et al. Prompt Optimization with Logged Bandit Data. [arXiv:2504.02646](#), 2025.
- [3] Chen Wang et al. Automating Personalization: Prompt Optimization for Recommendation Reranking. [arXiv:2504.03965](#), 2025.
- [4] Lakshya A. Agrawal et al. GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning. [arXiv:2507.19457](#), 2025.
- [5] Zhaoxuan Tan et al. Democratizing Large Language Models via Personalized Parameter-Efficient Fine-Tuning. [EMNLP](#), 2024.
- [6] Seoyeon Kim and Jaehyung Kim. SPRInG: Continual LLM Personalization via Selective Parametric Adaptation and Retrieval-Interpolated Generation. [arXiv:2601.09974](#), 2026.
- [7] Jack Lanchantin et al. Bridging Offline and Online Reinforcement Learning for LLMs. [arXiv:2506.21495](#), 2025.